

SENG 326 Software Architecture Group Project

Task1 Architectural Style Recovery, Validation and Evaluation

To successfully complete this task, you are required to understand the functionality of Checkstyle 13.2.0 and the key relationships between its components. This requires downloading and understanding the source code and running the application.

Checkstyle has: one primary architecture style and a set of secondary architecture styles. You are required to recover and identify the **primary architecture and one secondary architecture**.

Part A: Architectural Recovery and Validation

- You are required to recover, validate and document the software architecture of the Checkstyle 13.2.0 application.
- You must infer the architecture style

Step 2.3: Infer the primary and secondary architecture

Based on the previous steps: infer:

- architectural elements, responsibilities of each element
- the primary and secondary architectural styles
- All must be justified using the evidence from previous steps.

Output:

- Description of inferred architectural elements and justifications.
- This should include code fragments, code examples, tool outputs and screenshots at the appropriate level of detail to support the text.
- Clearly show the path from code to inferred architecture.

Step 2.4: Validate and Prove the Architecture

The recovered architectures in Step 2.3 must be validated against the actual code.

Validation requires:

- Formally defining the inferred architecture
- Using a tool to check that the code conforms to the architecture
- Identifying any violations or inconsistencies: use this to refine your inferred architecture (and validate again) or to justify why you wish to ignore these violations.

Tools:

- JQAssistant: show query results validating the inferred architecture
- ArchUnit (conformance checking)

Output:

- Validation results, including the defined architectural rules, detected violations (if any), explanations or refinements.

Step 2.5: C4 Diagrams

Based on the validated architecture: produce C4 diagrams for the recovered primary and secondary architecture styles. Diagrams must reflect architectural understanding and be consistent with the validated architecture.

Output: C4 diagrams (code-level diagrams can be for selected components)

Tools:

- Structurizr DSL

Step 3: Lessons Learned

Report A: The previous steps and their results should be formatted into a report. Sufficient supporting information should be within the report and extra information should be provided in the Appendix and clearly referenced within the report. Any references and sources should be stated and clearly referenced in the report.

Supporting information includes: ArchUnit and Structurizer code, JQassistant queries, etc. This information should be sufficient for us to reproduce the steps in Task 1.

Part B: Performance and Scalability Analysis

You will evaluate the performance and scalability of Checkstyle by measuring its **response time** when analyzing Java codebases of increasing size, with **all Checkstyle checks enabled**.

You are to analyze the observed performance based on your understanding of the Checkstyle architectures recovered in Part A.

Input Requirements:

- Five Java open source codebases of increasing size.
- Codebases must differ primarily in size (e.g., lines of code, number of files).
- Sizes must be chosen such that:
 - at least **two data points lie beyond the “knee”** of the curve (i.e., where response time growth becomes non-linear).
- You must clearly report:
 - how size is measured (e.g., LOC, number of files),
 - why the selected sizes are appropriate.

Measurement Procedure:

For each Java codebase:

1. Run Checkstyle using the same configuration.
2. Measure total response time.
3. Repeat run 3 times to reduce noise.
4. Record the mean response time of the 3 runs and calculate confidence intervals.

All runs for all codebases must be performed under the same conditions.

Plotting

Plot a graph of the mean response time versus codebase size.

The graph must clearly indicate:

- the five data points,
- the region where the “knee” occurs.
- the confidence intervals.

Analysis

Analyse the performance and scalability of Checkstyle based on the experiment result and your understanding of the recovered architecture.

Lessons Learned

Report Part B: Your report should contain all the above and the following:

- Description of the selected codebases and their sizes + links to the source code
- Clear description of the experimental setup: laptop H/W, any background applications (preferably nothing at the same time), etc.
- Clear description of the execution of the experiment: e.g., how did you measure the response time
- The analysis of the results
- Lessons Learned

Deadline for Task 1: Saturday March 28th 12 midnight.